

Note: - This assignment only for students, not for sell or re-upload any media or website. All right reserve to "IGNOU Study Helper". It is illegal to share or reupload it. If anything like this is found, then appropriate action will be taken and apply copyright ACT to you. You will be responsible for illegal work. So don't share and upload on any media.

Q1: a) Design and develop an efficient algorithm to find the list of prime numbers in the range 501 to 2000. What is the complexity of this algorithm?

Ans. To efficiently find all prime numbers in the range 501 to 2000, we can use the Sieve of Eratosthenes algorithm with some optimizations. The Sieve of Eratosthenes is one of the most efficient ways to find all prime numbers up to a given limit. Below is the step-by-step approach:

Algorithm:

1. Initialize a Boolean Array:

- a) Create an array `isPrime[]` of size $2000 + 1$ and set all values to true.
- b) `isPrime[i] = true` means i is assumed to be prime.

2. Mark Non-Prime Numbers:

- a) Iterate from 2 to $\sqrt{2000}$ and mark all multiples of each prime as false in the `isPrime[]` array.

3. Extract Primes in the Given Range (501 to 2000):

- a) Iterate through the range 501 to 2000 and collect numbers that remain true in the `isPrime[]` array.

Code Implementation:

```
def sieve_of_eratosthenes(start, end):
```

```
    n = end
```

```
    is_prime = [True] * (n + 1)
```

```
    is_prime[0] = is_prime[1] = False # 0 and 1 are not prime
```

```
    for i in range(2, int(n ** 0.5) + 1):
```

```
        if is_prime[i]:
```

```
            for j in range(i * i, n + 1, i):
```

```
                is_prime[j] = False
```

```
# Collect primes in the range [start, end]
```

```
primes = [i for i in range(start, end + 1) if is_prime[i]]
```

```
return primes
```

```
# Find primes in the range 501 to 2000
```

```
prime_numbers = sieve_of_eratosthenes(501, 2000)
```

```
print(prime_numbers)
```

Output:

```
[503, 509, 521, 523, 541, 547, 557, 563, 569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 631, 641, 643, 647, 653, 659, 661, 673, 677, 683, 691, 701, 709, 719, 727, 733, 739, 743, 751, 757, 761, 769, 773, 787, 797, 809, 811, 821, 823, 827, 829, 839, 853, 857, 859, 863, 877, 881, 883, 887, 907, 911, 919, 929, 937, 941, 947, 953, 967, 971, 977, 983, 991, 997, 1009, 1013, 1019, 1021, 1031, 1033, 1039, 1049, 1051, 1061, 1063, 1069, 1087, 1091, 1093, 1097, 1103, 1109, 1117, 1123, 1129, 1151, 1153, 1163, 1171, 1181, 1187, 1193, 1201, 1213, 1217, 1223, 1229, 1231, 1237, 1249, 1259, 1277, 1279, 1283, 1289, 1291, 1297, 1301, 1303, 1307, 1319, 1321, 1327, 1361, 1367, 1373, 1381, 1399, 1409, 1423, 1427, 1429, 1433, 1439, 1447, 1451, 1453, 1459, 1471, 1481, 1483, 1487, 1489, 1493, 1499, 1511, 1523, 1531, 1543, 1549, 1553, 1559, 1567, 1571, 1579, 1583, 1597, 1601, 1607, 1609, 1613, 1619, 1621, 1627, 1637, 1657, 1663, 1667, 1669, 1693, 1697, 1699, 1709, 1721, 1723, 1733, 1741, 1747, 1753, 1759, 1777, 1783, 1787, 1789, 1801, 1811, 1823, 1831, 1847, 1861, 1867, 1871, 1873, 1877, 1879, 1889, 1901, 1907, 1913, 1931, 1933, 1949, 1951, 1973, 1979, 1987, 1993, 1997, 1999]
```

=== Code Execution Successful ===



Time Complexity Analysis

1. Preprocessing (Marking Non-Primes):

- The sieve iterates up to $\sqrt{N}$, and for each prime, it marks multiples.
- This results in a complexity of $O(N \log \log N)$.

2. Extracting Primes in the Range:

- This takes $O(M)$, where M is the number of primes in the range.

Overall Complexity: $O(N \log \log N)$

b) Differentiate between Cubic-time and Factorial-time algorithms. Give example of one algorithm each for these two running times.

Ans. Difference Between Cubic-Time and Factorial-Time Algorithms

Aspect	Cubic-Time ($O(n^3)$)	Factorial-Time ($O(n!)$)
Definition	The running time grows proportional to n^3 .	The running time grows proportional to $n!$.

Growth Rate	Moderate growth, manageable for large n .	Extremely fast growth, infeasible for large n .
Efficiency	More efficient than factorial time but still slow for very large inputs.	Extremely inefficient for large inputs.
Typical Use Case	Algorithms dealing with 3D matrices, triplet combinations, or nested loops.	Algorithms that require generating all possible permutations or subsets.
Example Algorithm	Matrix Multiplication (Naïve Algorithm) – $O(n^3)$	Travelling Salesman Problem (Brute Force) – $O(n!)$

Example Algorithm for Cubic-Time Complexity ($O(n^3)$):

Naïve Matrix Multiplication

The classical matrix multiplication algorithm computes the product of two $n \times n$ matrices using three nested loops.

Python Code:

```
def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)] # Resultant matrix

    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]

    return C

# Example
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
result = matrix_multiplication(A, B)
print(result)
```

Output:

```
[[19, 22], [43, 50]]
```

```
=== Code Execution Successful ===
```

Time Complexity: $O(n^3)$ due to three nested loops.

Example Algorithm for Factorial-Time Complexity ($O(n!)$):

Brute Force Travelling Salesman Problem (TSP)

The TSP problem requires finding the shortest route that visits all cities exactly once and returns to the starting city. A brute-force approach generates all possible city orderings ($n!$ permutations) and finds the shortest one.

Python Code:

```
def travelling_salesman(graph):  
    n = len(graph)  
    cities = list(range(n))  
    min_cost = float('inf')  
  
    for perm in permutations(cities): # Generate all possible orderings (O(n!))  
        cost = sum(graph[perm[i]][perm[i+1]] for i in range(n-1))  
        cost += graph[perm[-1]][perm[0]] # Return to the start  
        min_cost = min(min_cost, cost)  
  
    return min_cost  
  
# Example Graph (Adjacency Matrix)  
graph = [  
    [0, 10, 15, 20],  
    [10, 0, 35, 25],  
    [15, 35, 0, 30],  
    [20, 25, 30, 0]  
]  
  
print(travelling_salesman(graph))
```

Output:

80

=== Code Execution Successful ===

Time Complexity: $O(n!)$ since all $n!$ permutations of cities are considered.**c) Write an algorithm to multiply two square matrices of order $n \times n$. Also explain the time complexity of this algorithm.****Ans.** We will use the naïve matrix multiplication algorithm, which follows a brute-force approach with three nested loops.**Algorithm: Naïve Matrix Multiplication****Steps:**

1. Create a result matrix C of size $n \times n$ initialized with zeros.
2. Iterate through each row of the first matrix A.
3. Iterate through each column of the second matrix B.
4. Compute the dot product of the corresponding row of A and the column of B, storing the sum in $C[i][j]$.
5. Return the resultant matrix C.


```
def matrix_multiplication(A, B):  
    n = len(A) # Get the order of the matrices  
    C = [[0] * n for _ in range(n)] # Initialize result matrix  
  
    for i in range(n):  
        for j in range(n):  
            for k in range(n):  
                C[i][j] += A[i][k] * B[k][j] # Compute dot product  
  
    return C  
  
# Example Usage  
A = [[1, 2], [3, 4]]  
B = [[5, 6], [7, 8]]  
result = matrix_multiplication(A, B)  
  
# Display result  
for row in result:  
    print(row)
```

Output:

[19, 22]

[43, 50]

=== Code Execution Successful ===

Time Complexity Analysis

1. The algorithm consists of three nested loops:

- Outer loop:** Iterates n times (for rows of A).
- Middle loop:** Iterates n times (for columns of B).
- Inner loop:** Iterates n times (for multiplication and summation).

2. Since each loop runs $O(n)$ times, the total number of operations is:

$$O(n) \times O(n) \times O(n) = O(n^3)$$

3. Final Complexity: $O(n^3)$

d) What are asymptotic bounds for analysis of efficiency of algorithms? Why are asymptotic bounds used? What are their shortcomings? Explain the Big O and Big Θ notation with the help of a diagram. Find the Big O-notation and Θ -notation for the function:

$$f(n) = 100n^4 + 1000n^3 + 100000$$

Ans. Asymptotic Bounds: Asymptotic bounds describe how the running time or space used by an algorithm grows with the size of the input n . They focus on the most significant factors (i.e., the highest-order term) as n becomes large.

Asymptotic Bounds Used:

- Machine-Independent Analysis:** They allow us to analyze algorithms without worrying about machine-specific constants or low-level optimizations.
- Focus on Dominant Behavior:** By ignoring constant factors and lower-order terms, asymptotic bounds emphasize how the algorithm scales with large inputs.
- Comparative Analysis:** They help in comparing different algorithms on a theoretical basis to decide which one will perform better as the input size increases.

Shortcomings of Asymptotic Bounds:

- Ignores Constant Factors:** For small input sizes, constant factors and lower-order terms can be significant, but asymptotic analysis does not capture these details.
- Not Always Reflective of Practical Performance:** Two algorithms with the same asymptotic bound might have very different actual running times due to implementation differences, memory access patterns, or hardware considerations.
- Worst-Case Focus:** Often, the analysis is performed for the worst-case scenario. However, the average-case or best-case might be more relevant in practice.

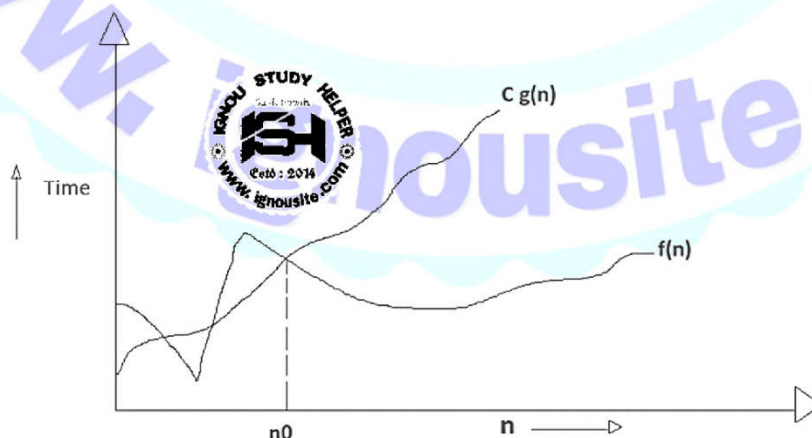
Big O notation (O):

It is defined as upper bound and upper bound on an algorithm is the most amount of time required (the worst case performance).

Big O notation is used to describe the asymptotic upper bound.

Mathematically, if $f(n)$ describes the running time of an algorithm; $f(n)$ is $O(g(n))$ if there exist positive constant C and n_0 such that,

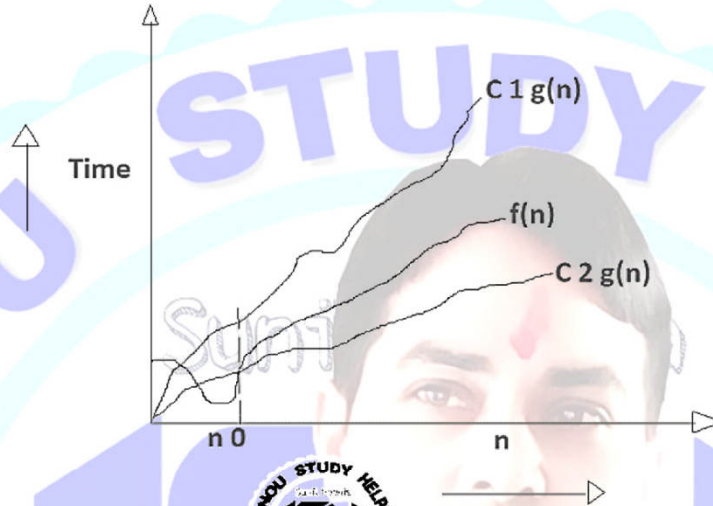
$$0 \leq f(n) \leq Cg(n) \text{ for all } n \geq n_0$$



It is defined as tightest bound and tightest bound is the best of all the worst case times that the algorithm can take.

Let $f(n)$ define running time of an algorithm.

$f(n)$ is said to be $\Theta(g(n))$ if $f(n)$ is $O(g(n))$ and $f(n)$ is $\Omega(g(n))$.



Analyzing $f(n)=100n^4+1000n^3+100000$

1. Highest-Degree Term:

The dominant term is $100n^4$, which will outweigh the lower-order terms $1000n^3$ and 100000 for large n .

2. Big O Notation:

- Since the function is dominated by n^4 , we have: $f(n)=O(n^4)$
- There exists a constant c (for example, $c=100+\epsilon$ for sufficiently large n) such that $f(n)\leq c\cdot n^4$.

3. Big Theta Notation:

- Because $f(n)$ is asymptotically sandwiched between two constant multiples of n^4 :
 $f(n)=\Theta(n^4)$
- This means that for large n , $f(n)$ grows at the same rate as n^4 .

e) Write and explain the Left to Right binary exponentiation algorithm. Demonstrate the use of this algorithm to compute the value of 3^{29} (Show the steps of computation). Explain the worst-case complexity of this algorithm.

Ans. Left to Right Binary Exponentiation Algorithm:

Binary exponentiation is a fast way to compute powers by breaking down the exponent into binary representation and using repeated squaring and multiplication. The Left to Right binary exponentiation algorithm processes the exponent from the most significant bit (MSB) to the least significant bit (LSB).

Algorithm:

- Convert the exponent n into its binary representation.
- Initialize result = 1.
- Traverse the bits of n from left to right (MSB to LSB):
 - Square the current result.
 - If the current bit is 1, multiply the result by the base.

4. Return the final result.

Compute 3^{29} using Left to Right Binary Exponentiation

Step 1: Convert 29 to Binary

$$29_{10} = 11101_2$$

(Starting from the leftmost bit: $1 \rightarrow 1 \rightarrow 1 \rightarrow 0 \rightarrow 1$)

Step 2: Initialize

result=1, base=3

Step 3: Traverse the binary exponent

First bit (1):

$$\text{result} = 1 \times 3 = 3$$

Next bit (1):

$$\text{Square result: } 3^2 = 9$$

$$\text{Multiply by base: } 9 \times 3 = 27$$

Next bit (1):

$$\text{Square result: } 27^2 = 729$$

$$\text{Multiply by base: } 729 \times 3 = 2187$$

Next bit (0):

$$\text{Square result: } 2187^2 = 4782969$$

(No multiplication since the bit is 0)

Next bit (1):

$$\text{Square result: } 4782969^2 = 22876792454961$$

$$\text{Multiply by base: } 22876792454961 \times 3 = 68630377364883$$

Final Result

$$3^{29} = 68630377364883$$

- We traverse all $\log(n)$ bits of the exponent.
- Each bit requires one squaring operation.
- If the bit is 1, an additional multiplication is performed.
- In the worst case (when every bit is 1), we perform $\log(n)$ squarings + $\log(n)$ multiplications.
- Each squaring and multiplication is $O(1)$ in basic arithmetic but takes $O(\log n)$ for large numbers.

Thus, the worst-case complexity is:

$O(\log n)$

This makes binary exponentiation significantly faster than naive exponentiation $O(n)$.

f) Write and explain the Bubble sort algorithm. Discuss its best and worst-case time complexity.

Ans. Bubble Sort Algorithm: Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The process repeats until the list is sorted.

Algorithm (Step-by-Step)

- Start from the first element in the array.
- Compare the current element with the next element.
- If the current element is greater than the next element, swap them.
- Move to the next pair and repeat the process until the last element.
- Repeat the process for the remaining elements, ignoring the last sorted elements in each pass.
- Continue this process until no swaps are needed, meaning the array is sorted.

Code:

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        swapped = False  
        for j in range(n - i - 1):  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
                swapped = True  
        if not swapped:  
            break
```

```
arr = [64, 34, 25, 12, 22, 11, 90]
```

```
bubble_sort(arr)
```

```
print("Sorted array:", arr)
```

Time Complexity Analysis

Best-Case Time Complexity ($O(n)$)

- If the array is already sorted, Bubble Sort will still run through the list once.
- The inner loop will not perform any swaps, and the algorithm will terminate early using the swapped flag.
- Thus, in the best case, the time complexity is $O(n)$ (linear time).

Worst-Case Time Complexity ($O(n^2)$)

- If the array is in reverse order, Bubble Sort will perform the maximum number of comparisons and swaps.
- It will go through $n-1$ passes, making $(n-1) + (n-2) + \dots + 1 = O(n^2)$ comparisons.
- Hence, the worst-case complexity is $O(n^2)$ (quadratic time).

g) What are the uses of recurrence relations? Solve the following recurrence relations using the Master's method

a. $T(n) = 4T(n/4) + n^1$

b. $T(n) = 4T(3n/4) + n^1$

Ans. Uses of Recurrence Relations:

Recurrence relations are widely used in computer science and mathematics to describe problems that have recursive structures. Some key uses include:

- Algorithm Analysis:** Recurrences help analyze the time complexity of recursive algorithms like Merge Sort, Quick Sort, and Divide & Conquer problems.
- Dynamic Programming:** Used to define optimal substructure properties in problems such as Fibonacci sequences and matrix chain multiplication.
- Combinatorics:** Helps in counting problems, such as finding the number of ways to arrange objects.
- Computer Graphics:** Used in recursive rendering algorithms like fractals.
- Data Structures:** Helps in analyzing tree-based structures like Binary Search Trees (BST) and heaps.

Solving Recurrence Relations Using Master's Theorem

The Master's Theorem solves recurrence relations of the form:

$$T(n) = aT(n/b) + f(n)$$

where:

- a = Number of subproblems
- b = Factor by which the problem size reduces
- $f(n)$ = Additional work done outside the recursive calls

The complexity is determined by comparing $f(n)$ with $O(n^{\log_b a})$ (which is the work done by recursive calls). The three cases are:

1. **Case 1:** If $f(n)=O(n^c)$ where $c < \log_b a$, then $T(n)=O(n^{\log_b a})$.
2. **Case 2:** If $f(n)=O(n^{\log_b a})$, then $T(n)=O(n^{\log_b a} \log n)$.
3. **Case 3:** If $f(n)=\Omega(n^c)$ where $c > \log_b a$, then $T(n)=O(f(n))$ (if a regularity condition holds).

(a) $T(n)=4T(n/4)+n^1$

Comparing with the Master's Theorem:

- a. $a=4, b=4, f(n)=n^1$.
- b. Compute $\log_b a = \log_4 4 = 1$.

Since $f(n)=n^1=O(n^{\log_4 4})$, we fall into Case 2.

Thus, $T(n)=O(n^1 \log n)=O(n \log n)$.

(b) $T(n)=4T(3n/4)+n^1$

Comparing with the Master's Theorem:

- a. $a=4, b=3/4, f(n)=n^1$.
- b. Compute $\log_{3/4} 4$.

$$\log_{3/4} 4 = \log 4 / \log(4/3)$$

Since this value is greater than 1, we compare it with $f(n)=O(n^1)$.

Since n^1 grows slower than $n^{\log_{3/4} 4}$, we apply Case 1.

Thus, $T(n)=O(n^{\log_{3/4} 4})$.

Q2: a) What is an Optimisation Problem? Explain with the help of an example. When would you use a Greedy Approach to solve optimisation problem? Formulate the Task Scheduling Problem as an optimisation problem and write a greedy algorithm to solve this problem. Also, solve the following fractional Knapsack problem using greedy approach. Show all the steps.

Suppose there is a knapsack of capacity 20 Kg and the following 6 items are to be packed in it. The weight and profit of the items are as under:

$(p_1, p_2, \dots, p_6) = (30, 16, 18, 20, 10, 7)$

$(w_1, w_2, \dots, w_6) = (5, 4, 6, 4, 5, 7)$

Select a subset of the items that maximises the profit while keeping the total weight below or equal to the given capacity.

Ans. Optimisation Problem: An Optimisation Problem is a problem where we need to find the best solution from a set of possible solutions while satisfying given constraints. The objective could be maximizing or minimizing a function.

Example:

Consider a traveling salesman problem (TSP): A salesman must visit multiple cities and return to the starting city while minimizing the total travel distance. The objective is to find the shortest possible route that covers all cities exactly once.

A Greedy Algorithm is used when:

1. The problem exhibits the Greedy Choice Property, meaning local optimal choices lead to a globally optimal solution.
2. The problem follows Optimal Substructure, meaning an optimal solution to the problem contains optimal solutions to its subproblems.

A Greedy Algorithm makes a sequence of choices, each appearing best at the moment, to reach the optimal solution.

Task Scheduling Problem as an Optimisation Problem

Problem Formulation:

Given n tasks with a deadline and profit, schedule the tasks to maximize total profit while ensuring each task is completed before its deadline.

Input:

- A set of tasks $T = \{T_1, T_2, \dots, T_n\}$
- Each task T_i has a deadline d_i and a profit p_i
- Only one task can be executed at a time

Objective:

Maximize the total profit while completing tasks within their deadlines.

Greedy Algorithm for Task Scheduling

1. Sort all tasks in decreasing order of profit.
2. Initialize an array for scheduling tasks, with slots from 1 to max deadline.
3. Iterate through each task and place it in the latest available slot before its deadline.
4. If a slot is available, assign the task to that slot.
5. Continue until all tasks are scheduled or slots are filled.

Greedy Algorithm Implementation

```
def task_scheduling(tasks):
```

```
# Step 1: Sort tasks by profit in descending order
tasks.sort(key=lambda x: x[1], reverse=True)
```

```
max_deadline = max(task[2] for task in tasks)
```

```
schedule = [-1] * (max_deadline + 1) # Array to keep track of scheduled tasks
```

```
total_profit = 0
```

```
for task in tasks:
```

```
    task_name, profit, deadline = task
```

```
    for j in range(min(deadline, max_deadline), 0, -1):
```

```
        if schedule[j] == -1: # If the slot is available
```

```
            schedule[j] = task_name
```

```
            total_profit += profit
```

```
            break
```


return schedule, total_profit

```
# Example tasks: (Task Name, Profit, Deadline)
tasks = [("T1", 100, 2), ("T2", 50, 1), ("T3", 200, 2), ("T4", 20, 1)]
schedule, max_profit = task_scheduling(tasks)
print("Scheduled Tasks:", schedule)
print("Maximum Profit:", max_profit)
```

Fractional Knapsack Problem using Greedy Approach

Given Data

- Knapsack Capacity = 20 kg
- Items (Profit, Weight): (p1,w1)=(30,5), (p2,w2)=(16,4), (p3,w3)=(18,6), (p4,w4)=(20,4), (p5,w5)=(10,5), (p6,w6)=(7,7)

Step 1: Compute Profit-to-Weight Ratio

$$\frac{p_1}{w_1} = \frac{30}{5} = 6, \quad \frac{p_2}{w_2} = \frac{16}{4} = 4, \quad \frac{p_3}{w_3} = \frac{18}{6} = 3, \quad \frac{p_4}{w_4} = \frac{20}{4} = 5, \quad \frac{p_5}{w_5} = \frac{10}{5} = 2, \quad \frac{p_6}{w_6} = \frac{7}{7} = 1$$

Step 2: Sort Items by Profit-to-Weight Ratio

(p1,6), (p4,5), (p2,4), (p3,3), (p5,2), (p6,1)

Step 3: Pick Items Greedily

Step	Item	Weight Taken	Remaining Capacity	Profit Gained
1	p ₁	5 kg	15 kg	30
2	p ₄	4 kg	11 kg	20 + 30 = 50
3	p ₂	4 kg	7 kg	16 + 50 = 66
4	p ₃	6 kg	1 kg	18 + 66 = 84
5	p ₅	1 kg (fraction)	0 kg	$\frac{10}{5} \times 1 = 2$

Final Solution

- Total Profit = 84 + 2 = 86
- Selected Items: p1, p4, p2, p3 and part of p5

b) Assuming that data to be transmitted consists of only characters 'a' to 'g', design the Huffman code for the following frequencies of character data. Show all the steps of building a Huffman tree. Also, show how a coded sequence using Huffman code can be decoded.

a:5, b:25, c:10, d:15, e:8, f:7, g:30

Ans. Huffman Coding for Given Frequencies: Huffman coding is a lossless data compression technique that assigns variable-length binary codes to input characters, with shorter codes for more frequent characters.

Given character frequencies:

a:5, b:25, c:10, d:15, e:8, f:7, g:30

Step 1: Build Huffman Tree

Step 1.1: Create a Min Heap

We start by inserting all characters into a priority queue (min heap), where the lowest frequency nodes have the highest priority.

Character	Frequency
a	5
b	25
c	10
d	15
e	8
f	7
g	30

Step 1.2: Build Tree by Merging Lowest Frequency Nodes

1. Merge nodes (a,5) and (f,7) → New node (af, 5+7=12)

Heap:

(b,25), (c,10), (d,15), (e,8), (g,30), (af,12)

2. Merge nodes (e,8) and (c,10) → New node (ec, 8+10=18)

Heap:

(b,25), (d,15), (g,30), (af,12), (ec,18)

3. Merge nodes (af,12) and (d,15) → New node (afd, 12+15=27)

Heap:

(b,25), (g,30), (ec,18), (afd,27)

4. Merge nodes (b,25) and (ec,18) → New node (b-ec, 25+18=43)

Heap:

5. Merge nodes (afd,27) and (g,30) → New node (afd-g, 27+30=57)

Heap:

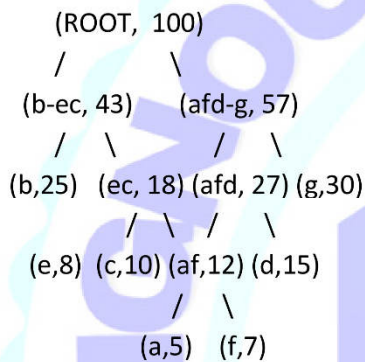
(b-ec,43),(afd-g,57)

6. Merge nodes (b-ec,43) and (afd-g,57) → New node (ROOT, 43+57=100)

Now, we have our Huffman Tree!

Step 2: Assign Huffman Codes

We assign 0 to the left child and 1 to the right child recursively.



Step 2.1: Generate Codes

Character	Huffman Code
g	11
d	101
f	1001
a	1000
b	00
e	010
c	011

Step 3: Encode a Sequence

Let's encode "bad" using our Huffman codes:

b → 00

a → 1000

d → 101

Encoded Sequence:

00 1000 101

Suppose we receive the encoded sequence:

"1011000011"

1. Read '1' → Check Tree → Not a leaf node → Read next '0' → 10 (Still not a leaf) → Read next '1' → 101 → d
2. Read '1' → Next '1' → 11 → g
3. Read '0' → Next '0' → 00 → b
4. Read '0' → Next '1' → Next '0' → 010 → e
5. Read '0' → Next '1' → Next '1' → 011 → c

Decoded Output: d g b e c

Final Huffman Codes

Character	Code
a	1000
b	00
c	011
d	101
e	010
f	1001
g	11

This method optimally compresses data and ensures efficient decoding.

c) Explain the Merge procedure of the Merge Sort algorithm. Demonstrate the use of recursive Merge sort algorithm for sorting the following data of size 8: [19, 18, 16, 12, 11, 10, 9, 8]. Compute the complexity of Merge Sort algorithm.

Ans. Merge Procedure in Merge Sort: Merge Sort is a divide-and-conquer algorithm that recursively divides the array into smaller subarrays, sorts them, and merges them back together in sorted order.

Merge Procedure

The Merge function takes two sorted subarrays and combines them into a single sorted array.

Steps of Merge Procedure:

1. Initialize Pointers:
Use three pointers: one for the left subarray, one for the right subarray, and one for the main array.
2. Compare Elements:
Compare the elements of both subarrays.
Copy the smaller element to the main array and move the corresponding pointer.
3. Copy Remaining Elements:
If one subarray is exhausted, copy the remaining elements from the other subarray.
4. Repeat Until Fully Merged.

Merge Sort recursively divides the array into two halves, sorts each half, and then merges them.

Recursive Merge Sort Algorithm

```
def merge(arr, left, mid, right):  
    left_subarray = arr[left:mid + 1] # Copy left subarray  
    right_subarray = arr[mid + 1:right + 1] # Copy right subarray  
  
    i, j, k = 0, 0, left # Initialize indices  
  
    # Merge two sorted subarrays  
    while i < len(left_subarray) and j < len(right_subarray):  
        if left_subarray[i] <= right_subarray[j]:  
            arr[k] = left_subarray[i]  
            i += 1  
        else:  
            arr[k] = right_subarray[j]  
            j += 1  
        k += 1  
  
    # Copy remaining elements of left subarray  
    while i < len(left_subarray):  
        arr[k] = left_subarray[i]  
        i += 1  
        k += 1  
  
    # Copy remaining elements of right subarray  
    while j < len(right_subarray):  
        arr[k] = right_subarray[j]  
        j += 1  
        k += 1  
  
def merge_sort(arr, left, right):  
    if left < right:  
        mid = (left + right) // 2 # Find middle  
        merge_sort(arr, left, mid) # Sort left half  
        merge_sort(arr, mid + 1, right) # Sort right half  
        merge(arr, left, mid, right) # Merge sorted halves  
  
# Given array  
arr = [19, 18, 16, 12, 11, 10, 9, 8]  
merge_sort(arr, 0, len(arr) - 1)  
print("Sorted Array:", arr)
```

Input Array:

[19,18,16,12,11,10,9,8]

Step-by-Step Execution**1. Divide Step:**

Split into two halves:

Left: [19, 18, 16, 12]**Right:** [11, 10, 9, 8]**2. Recursively Divide:**

Left → [19, 18] and [16, 12]

Right → [11, 10] and [9, 8]

3. Sort Individual Pairs:

[19, 18] → [18, 19]

[16, 12] → [12, 16]

[11, 10] → [10, 11]

[9, 8] → [8, 9]

4. Merge Pairs:

[18, 19] and [12, 16] → [12, 16, 18, 19]

[10, 11] and [8, 9] → [8, 9, 10, 11]

5. Final Merge:

[12, 16, 18, 19] and [8, 9, 10, 11]

Sorted Array: [8, 9, 10, 11, 12, 16, 18, 19]**Time Complexity of Merge Sort**

- **Divide Step:** Recursively split the array into halves → $O(\log n)$
- **Merge Step:** Merging two halves takes linear time → $O(n)$
- **Total Complexity:** $O(n \log n)$
(Since there are $O(\log n)$ levels, and merging takes $O(n)$ at each level)

d) Explain the divide and conquer approach of multiplying two large integers. Compute the time complexity of this approach. Also, explain the binary search algorithm and find its time complexity.

Ans. Divide and Conquer Approach for Multiplying Large Integers: The divide and conquer approach to multiplying large integers is often associated with Karatsuba's Algorithm. This algorithm reduces the multiplication of two n -digit numbers into smaller subproblems.

Algorithm Steps:**1. Split the numbers:** Split two large integers X and Y into two halves:

$$\text{➤ } X = 10^{n/2} \cdot A + B$$

$$\text{➤ } Y = 10^{n/2} \cdot C + D \text{ Here, } A, B, C, \text{ and } D \text{ represent smaller numbers, where } A \text{ and } C \text{ are the higher-order halves, and } B \text{ and } D \text{ are the lower-order halves.}$$

2. Multiply the parts:

➤ Compute three multiplications recursively:

- a) $P1=A \times C$ (high-order product)
- b) $P2=B \times D$ (low-order product)
- c) $P3=(A+B) \times (C+D) - P1 - P2$ (cross-product term)

3. Combine the results:

➤ The final product is: $X \times Y = P1 \times 10^n + P3 \times 10^{n/2} + P2$

Time Complexity:

- a) The problem size n is divided into two subproblems of size $n/2$.
- b) The recurrence relation is: $T(n) = 3T(n/2) + O(n)$
- c) Using the Master Theorem, this solves to: $T(n) = O(n^{\log_2 3}) \approx O(n^{1.585})$

This is faster than the naive $O(n^2)$ multiplication.

Binary Search Algorithm: Binary search is an efficient algorithm for finding the position of a target element in a sorted array. It works by repeatedly dividing the search interval in half.

Algorithm Steps:

1. **Initialize boundaries:** Set two pointers, $low=0$ and $high=n-1$, where n is the size of the array.
2. **Compute middle index:** Calculate $mid = \lfloor (low+high)/2 \rfloor$.
3. **Check the middle element:**
 - a) If the middle element matches the target, return its index.
 - b) If the middle element is greater than the target, search the left half by setting $high=mid-1$.
 - c) If the middle element is less than the target, search the right half by setting $low=mid+1$.
4. **Repeat:** Continue until the target is found or the search interval becomes empty.

Time Complexity:

- a) At each step, the search space is halved.
- b) The recurrence relation is: $T(n) = T(n/2) + O(1)$
- c) Solving this using the Master Theorem gives: $T(n) = O(\log n)$

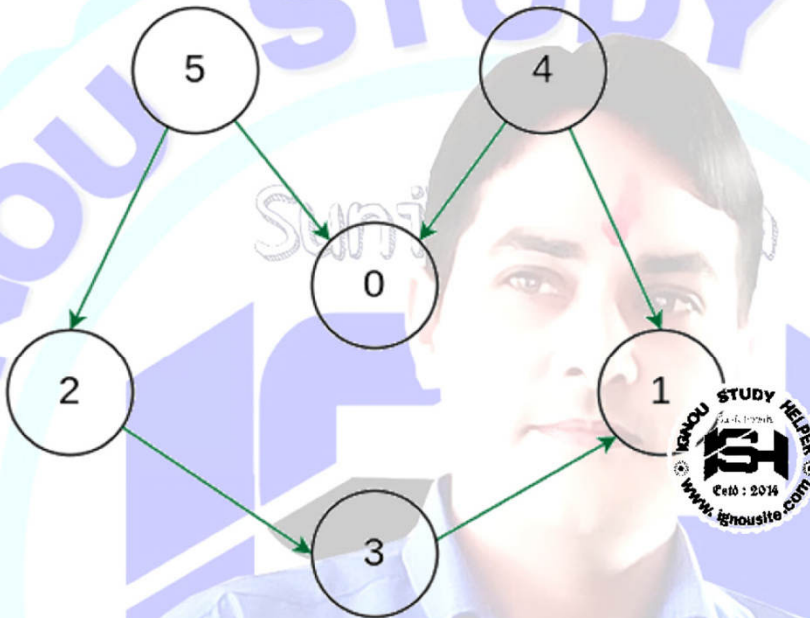
Binary search is very efficient and ideal for sorted datasets.

e) Explain the Topological sorting with the help of an example. Also, explain the algorithm of finding strongly connected components in a directed Graph.

Ans. Topological sorting: Topological sorting for Directed Acyclic Graph (DAG) is a linear ordering of vertices such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering.

Example:

Input: Graph:



Output: 5 4 2 3 1 0

Explanation: The first vertex in topological sorting is always a vertex with an in-degree of 0 (a vertex with no incoming edges). A topological sorting of the following graph is "5 4 2 3 1 0". There can be more than one topological sorting for a graph. Another topological sorting of the following graph is "4 5 2 3 1 0".

Algorithm for Finding Strongly Connected Components (SCC): Strongly Connected Components (SCC) are subgraphs in which every vertex is reachable from every other vertex.

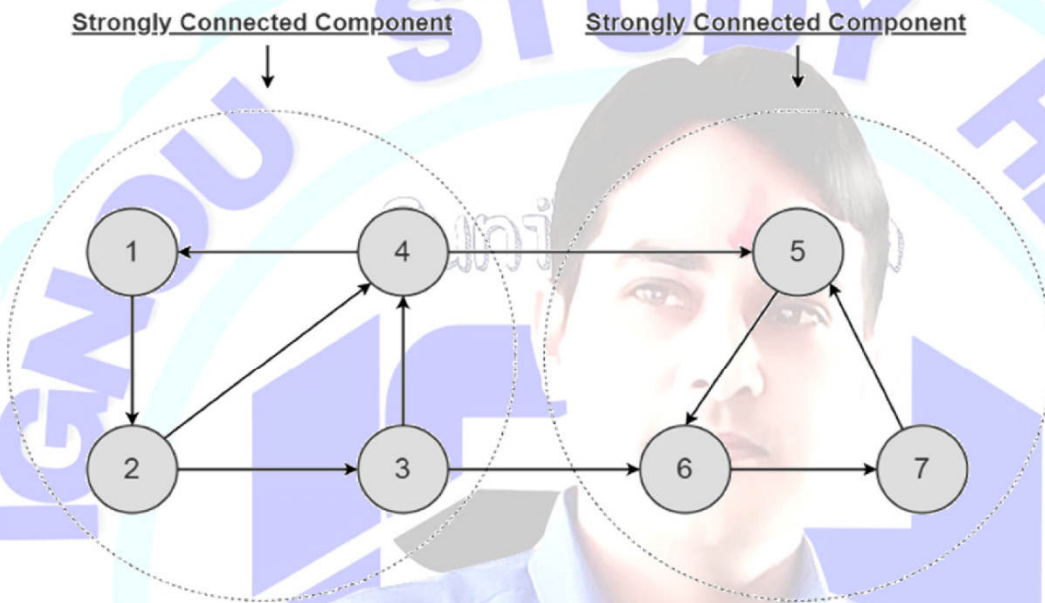
Steps to Find SCCs (Kosaraju's Algorithm)

1. **Step 1: Perform a Topological Order Traversal**
 - a) Do a Depth-First Search (DFS) on the graph.
 - b) Maintain a stack to store the finish time of vertices. Push vertices onto the stack in the order they finish.
2. **Step 2: Transpose the Graph**
 - a) Reverse the direction of all edges in the graph.
3. **Step 3: Process Vertices in Stack Order**
 - a) While the stack is not empty:
 - I. Pop a vertex from the stack.
 - II. Perform DFS starting from that vertex in the transposed graph.

- III. Collect all reachable vertices as a single SCC.
b) Repeat until the stack is empty.

Strongly Connected Components: A strongly connected component of a directed graph is a maximal subgraph where every pair of vertices is mutually reachable. This means that for any two nodes A and B in this subgraph, there is a path from A to B and a path from B to A.

For example: The below graph has two strongly connected components {1,2,3,4} and {5,6,7} since there is path from each vertex to every other vertex in the same strongly connected component.



Q3: Consider the following Graph:

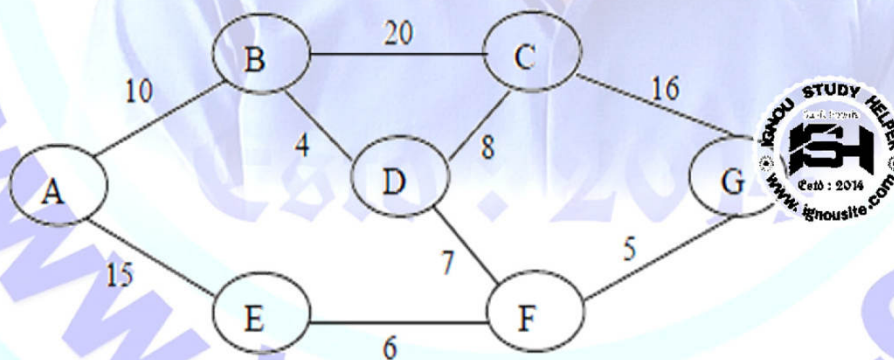


Figure 1: A sample weighted Graph

a) Write the Prim's algorithm to find the minimum cost spanning tree of a graph. Also, find the time complexity of Prim's algorithm. Demonstrate the use of Kruskal's algorithm and Prim's algorithm to find the minimum cost spanning tree for the Graph given in Figure 1. Show all the steps.

Ans. Prim's Algorithm: Prim's algorithm is a greedy algorithm that finds the Minimum Spanning Tree (MST) of a graph. It starts from a single vertex and adds the smallest edge that expands the tree without forming a cycle.

Steps for Prim's Algorithm

1. Initialize an empty MST.
2. Choose any vertex as the starting point.
3. Add the smallest edge connecting a vertex in the MST to a vertex outside the MST.
4. Repeat until all vertices are included in the MST.

Time Complexity of Prim's Algorithm

- a) **Using adjacency matrix and array:** $O(V^2)$, where V is the number of vertices.
- b) **Using priority queue (binary heap):** $O(E \log V)$, where E is the number of edges.

Kruskal's Algorithm: Kruskal's algorithm is another greedy algorithm for finding the MST. It sorts all edges by weight and adds them to the MST, ensuring no cycles.

Steps for Kruskal's Algorithm

1. Sort all edges in increasing order of their weights.
2. Initialize an empty MST.
3. Iterate over the sorted edges:
 - a) Add an edge to the MST if it does not form a cycle.
4. Repeat until $V-1$ edges are added.

Time Complexity of Kruskal's Algorithm

1. $O(E \log E)$, as sorting the edges dominates the computation.

Demonstration with Figure 1**Graph Details**

Vertices: A,B,C,D,E,F,G

Edges with weights:

A→B: 10

A→E: 15

B→D: 4

C→D: 8

C→G: 16

D→F: 7

B→C: 20

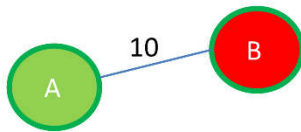
F→G: 5

Using Prim's Algorithm

1. Start at vertex A.
Include A. The edges available are:
 - a) A→B:10

b) $A \rightarrow E:15$

Choose $A \rightarrow B$ (smallest).

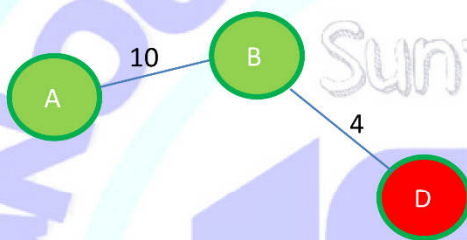


2. Include B. The edges available are:

a) $B \rightarrow D:4$

b) $A \rightarrow E:15$

Choose $B \rightarrow D$.



3. Include D. The edges available are:

a) $D \rightarrow F:7$

b) $C \rightarrow D:8$

c) $A \rightarrow E:15$

Choose $D \rightarrow F$.



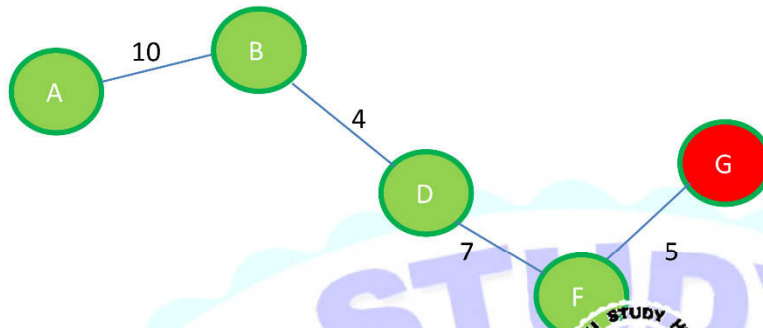
4. Include F. The edges available are:

a) $F \rightarrow G:5$

b) $C \rightarrow D:8$

c) $A \rightarrow E:15$

Choose $F \rightarrow G$

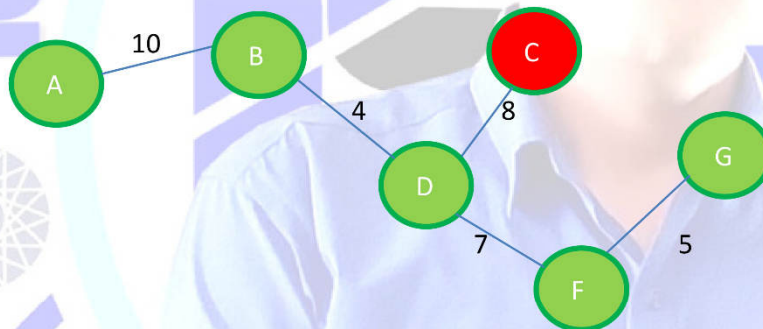


5. Include G. The edges available are:

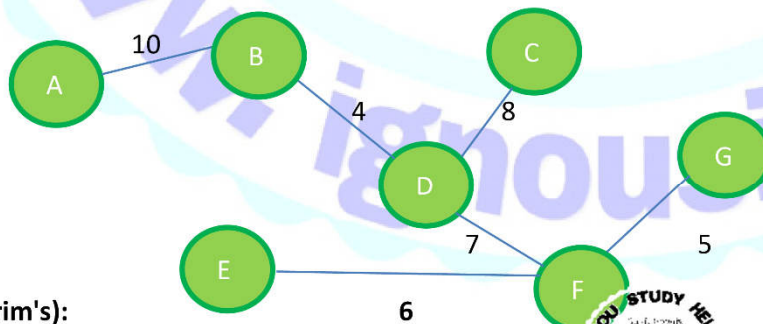
a) $C \rightarrow D: 8$

b) $F \rightarrow E: 6$

Choose $C \rightarrow D$.



6. Include C. The edge $F \rightarrow E$ is the only one left. Choose $F \rightarrow E$.



MST Edges (Prim's):

$A \rightarrow B, B \rightarrow D, D \rightarrow F, F \rightarrow G, C \rightarrow D, F \rightarrow E$.

1. Sort edges by weight:
 $B \rightarrow D:4, F \rightarrow G:5, D \rightarrow F:7, C \rightarrow D:8, A \rightarrow B:10, A \rightarrow E:15, C \rightarrow G:16, D \rightarrow C:20$.
2. Start with $B \rightarrow D:4$ Include it.
3. Add $F \rightarrow G:5$.
4. Add $D \rightarrow F:7$.
5. Add $C \rightarrow D:8$.
6. Add $A \rightarrow B:10$.
7. Add $F \rightarrow E:6$.

MST Edges (Kruskal's):

$B \rightarrow D, F \rightarrow G, D \rightarrow F, C \rightarrow D, A \rightarrow B, F \rightarrow E$.

Cost of MST (Both Algorithms): $4+5+7+8+10+6=40$.

b) Write the Dijkstra's shortest path algorithm. Also, find the time complexity of this shortest path algorithm. Find the shortest paths from the vertex 'A' using Dijkstra's shortest path algorithm for the graph given in Figure 1. Show all the steps of computation.

Ans. Dijkstra's Shortest Path Algorithm: Dijkstra's algorithm computes the shortest path from a source vertex to all other vertices in a weighted graph with non-negative edge weights.

Algorithm Steps

1. **Initialization:**
 - a) Assign a tentative distance of ∞ to all vertices except the source, which is set to 0.
 - b) Create a priority queue (or min-heap) to manage vertices based on their tentative distances.
2. **Process Vertices:**
 - a) Extract the vertex u with the smallest tentative distance from the priority queue.
 - b) For each neighbor v of u , calculate: tentative distance to $v = \text{distance}[u] + \text{weight}(u, v)$
 - c) If the new tentative distance is smaller than the current distance for v , update it and reinsert v into the priority queue.
3. **Repeat:**
 - a) Mark u as visited and do not process it again.
 - b) Continue until all vertices are visited or the priority queue is empty.
4. **Output:**
 - a) The shortest distances from the source to all vertices.

- a) Using an adjacency list and priority queue (binary heap): $O(E \log V)$, where E is the number of edges and V is the number of vertices.
- b) Using an adjacency matrix: $O(V^2)$.

Application to the Given Graph

Graph Details

Vertices: A,B,C,D,E,F,G

Edges with weights:

A→B: 10

A→E: 15

B→D: 4

D→C: 8

D→F: 7

C→G: 16

F→G: 5

Initialization

Start at A:

Distance: A=0, B=∞, C=∞, D=∞, E=∞, F=∞, G=∞.

Priority queue: {(A,0)}.

Step-by-Step Computation

1. Process A:

- a) Current distance: A=0.
- b) Update neighbors:
 - i. B: $0+10=10$,
 - ii. E: $0+15=15$.
- c) priority queue: {(B,10),(E,15)}.

2. Process B:

- a) Current distance: B=10.
- b) Update neighbors:
 - i. D: $10+4=14$.
- c) Priority queue: {(D,14),(E,15)}.

3. Process D:

- a) Current distance: D=14.
- b) Update neighbors:
 - i. C: $14+8=22$,
 - ii. F: $14+7=21$.

c) Priority queue: $\{(E,15),(F,21),(C,22)\}$.

4. Process E:

- a) Current distance: $E=15$.
- b) No updates (no unvisited neighbors).
- c) Priority queue: $\{(F,21),(C,22)\}$.

5. Process F:

- a) Current distance: $F=21$.
- b) Update neighbors:
 - i. $G:21+5=26$.
- c) Priority queue: $\{(C,22),(G,26)\}$.

6. Process C:

- a) Current distance: $C=22$.
- b) No updates (no unvisited neighbors).
- c) Priority queue: $\{(G,26)\}$.

7. Process G:

- a) Current distance: $G=26$.
- b) No updates (no unvisited neighbors).
- c) Priority queue: $\emptyset$.

Final Shortest Distances

- A: 0
- B: 10
- C: 22
- D: 14
- E: 15
- F: 21
- G: 26

Shortest Paths

- $A \rightarrow B: 10$
- $A \rightarrow B \rightarrow D: 14$
- $A \rightarrow B \rightarrow D \rightarrow C: 22$
- $A \rightarrow E: 15$
- $A \rightarrow B \rightarrow D \rightarrow F: 21$
- $A \rightarrow B \rightarrow D \rightarrow F \rightarrow G: 26$

c) Explain the algorithm to find the optimal Binary Search Tree. Demonstrate this algorithm to find the Optimal Binary Search Tree for the following probability data (where p_i represents the probability that the search will be for the key node k_i , whereas q_i represents that the search is for dummy node d_i . Make suitable assumptions, if any)

i	0	1	2	3	4
p_i		0.10	0.15	0.20	0.10
q_i	0.05	0.10	0.10	0.10	0.10

Ans.

Optimal Binary Search Tree (OBST) Algorithm: The Optimal Binary Search Tree minimizes the expected search cost for a Binary Search Tree, given:

1. Probabilities p_i for actual keys k_i .
2. Probabilities q_i for dummy keys (nonexistent keys between k_i).

Steps of the Algorithm

1. **Input:**
 - a) $p[i]$: Probability of searching for key k_i , where $1 \leq i \leq n$.
 - b) $q[i]$: Probability of searching for dummy node d_i where $0 \leq i \leq n$.
2. **Cost Matrix Initialization:**
 - a) Create two $(n+1) \times (n+1)$ matrices:
 - i. **Cost Matrix ($cost[i][j]$):** Stores the minimum search cost for the subtrees.
 - ii. **Weight Matrix ($weight[i][j]$):** Stores cumulative probabilities.
 - iii. Initialize $cost[i][i-1] = q[i-1]$ for $1 \leq i \leq n+1$.
3. **Dynamic Programming Table Construction:**
 - a) For subtree lengths $l=1$ to n :
 - i. For each subtree from $i=1$ to $n-l+1$:
 - i. Compute $j=i+l-1$.
 - ii. Compute cumulative probability: $weight[i][j] = weight[i][j-1] + p[j] + q[j]$
 - iii. Set $cost[i][j] = \infty$.
 - iv. Test each r (root between i and j) as the root: $temp_cost = cost[i][r-1] + cost[r+1][j] + weight[i][j]$
 - v. Update $cost[i][j]$ with the minimum $temp_cost$.
4. **Output:**
 - a) The value at $cost[1][n]$ gives the minimum search cost for the entire tree.
 - b) Use an auxiliary root matrix to reconstruct the tree structure.

Given Data

- i. Keys: k_1, k_2, k_3, k_4
- ii. Dummy keys: d_0, d_1, d_2, d_3, d_4
- iii. $p = [0.10, 0.15, 0.20, 0.10]$
- iv. $q = [0.05, 0.10, 0.10, 0.10, 0.10]$

Step 1: Initialization

- a) Initialize $\text{cost}[i][i-1] = q[i-1]$: $\text{cost}[1][0] = 0.05, \text{cost}[2][1] = 0.10, \dots, \text{cost}[5][4] = 0.10$
- b) Set $\text{weight}[i][j] = 0$ initially.

Step 2: Calculate Cumulative Probabilities and Costs

- a) For $l=1$:
Compute cost for each subtree of length 1:

$$\text{weight}[i][j] = q[i-1] + p[j] + q[j]$$

Example: For $i=1, j=1$:

$$\text{weight}[1][1] = q[0] + p[1] + q[1] = 0.05 + 0.10 + 0.10 = 0.25$$

$$\text{Update } \text{cost}[1][1] = \text{weight}[1][1] = 0.25$$

- b) For $l=2$:
Compute costs for subtrees of length 2.

Example: For $i=1, j=2$:

$$\text{weight}[1][2] = \text{weight}[1][1] + p[2] + q[2] = 0.25 + 0.15 + 0.10 = 0.50$$

Test roots k_1 and k_2 :

- i. For $r=1$: $\text{cost}[1][2] = \text{cost}[1][0] + \text{cost}[2][2] + \text{weight}[1][2]$
- ii. Repeat for $r=2$ and select the minimum cost.

Final Results

Using this process, compute for $l=3$ and $l=4$ until $\text{cost}[1][4]$ is derived.

Tree Structure

Reconstruct the tree using the root matrix and visualize the resulting OBST.

Time Complexity

- a) **Time Complexity:** $O(n^3)$, due to three nested loops (length, subtree, root).
- b) **Space Complexity:** $O(n^2)$, for storing cost and weight matrices.

d) Given the following sequence of chain multiplication of the matrices. Find the optimal way of multiplying these matrices:

Matrix	Dimension
A ₁	10 × 15
A ₂	15 × 5
A ₃	5 × 20
A ₄	20 × 10



Ans. To find the optimal way of multiplying matrices A₁, A₂, A₃, A₄ using the **Matrix Chain Multiplication** algorithm, we aim to minimize the number of scalar multiplications.

Input

Matrices and their dimensions:

A₁: 10×15

A₂: 15×5

A₃: 5×20

A₄: 20×10

Let p=[10,15,5,20,10], where p[i-1] and p[i] are the dimensions of the ith matrix.

Steps of the Algorithm

Step 1: Define m[i][j] and s[i][j]:

- m[i][j]: Minimum cost of multiplying matrices from A_i to A_j.
- s[i][j]: Index k that achieves the minimum cost for multiplying A_i to A_j.

Step 2: Initialize the cost matrix:

- For i=1 to n, m[i][i]=0, as multiplying a single matrix costs 0.

Step 3: Compute costs for chains of increasing lengths:

For each chain length l=2 to n:

- Compute m[i][j] for i=1 to n-l+1 and j=i+l-1.
- Test all possible values of k (i≤k<j) to split the chain:

$$m[i][j] = \min_{i \leq k < j} (m[i][k] + m[k+1][j] + p[i-1] \cdot p[k] \cdot p[j])$$

Computations

1. Initialization:

m[i][i]=0 for i=1,2,3,4

2. For chain length l=2:

- m[1][2]=10·15·5=750
- m[2][3]=15·5·20=1500
- m[3][4]=5·20·10=1000

3. For chain length $l=3$:

i. $m[1][3]$:

$$m[1][3] = \min(m[1][1] + m[2][3] + 10 \cdot 15 \cdot 20, m[1][2] + m[3][3] + 10 \cdot 5 \cdot 20) \\ = \min(0 + 1500 + 3000, 750 + 0 + 1000) = 1750$$

ii. $m[2][4]$:

$$m[2][4] = \min(m[2][2] + m[3][4] + 15 \cdot 5 \cdot 10, m[2][3] + m[4][4] + 15 \cdot 20 \cdot 10) \\ m[2][4] = \min(0 + 1000 + 750, 1500 + 0 + 3000) = 1750$$

4. For chain length $l=4$:

i. $m[1][4]$:

$$m[1][4] = \min(m[1][1] + m[2][4] + 10 \cdot 15 \cdot 10, m[1][2] + m[3][4] + 10 \cdot 5 \cdot 10, m[1][3] + m[4][4] + 10 \cdot 20 \cdot 10)$$

$$m[1][4] = \min(0 + 1750 + 1500, 750 + 1000 + 500, 1750 + 0 + 2000)$$

$$m[1][4] = \min(3250, 2250, 3750) = 2250$$

Result

The minimum cost to multiply A_1, A_2, A_3, A_4 is 2250 scalar multiplications.

Optimal Parenthesization

Using $s[i][j]$, reconstruct the solution:

Parenthesization: $((A_1 \cdot A_2) \cdot A_3) \cdot A_4$.

e) Explain the Rabin Karp algorithm for string matching with the help of an example. Find the time complexity of this algorithm.

Ans. Rabin-Karp Algorithm for String Matching: The Rabin-Karp Algorithm is a string matching algorithm that uses hashing to find a pattern within a text efficiently. It is particularly effective when we need to search for multiple patterns in a single text.

Algorithm Overview: The algorithm uses a rolling hash function to calculate the hash value of the pattern and substrings of the text. Instead of directly comparing substrings, it compares their hash values, which reduces the number of character comparisons.

Steps:

1. Calculate the hash value of the pattern (p) and the first substring of the text (t_0) of the same length.
2. Slide the pattern over the text one character at a time:
 - i. If the hash values match, compare the actual characters of the substring and pattern to confirm a match.
 - ii. If the hash values do not match, compute the hash value of the next substring using a rolling hash formula.
3. Repeat until the pattern is found or the text is fully scanned.

Hash Function: A hash function is used to convert a string into a numeric value. One common hash function for strings is:

$$h(s) = (s_0 \cdot d^{m-1} + s_1 \cdot d^{m-2} + \dots + s_{m-1}) \bmod q$$

Where:

- $s_0, s_1, \dots, s_{m-1}$ the characters of the string converted to integers.
- d : The base (e.g., 256 for extended ASCII).
- q : A prime number to avoid overflow.

Example

Input:

Text (T): "3141592653589793"

Pattern (P): "26535"

Base (d): 10

Prime (q): 13

Steps:

1. Calculate Initial Hash Values:

i. Pattern hash (h_p) = $(2 \cdot 10^4 + 6 \cdot 10^3 + 5 \cdot 10^2 + 3 \cdot 10^1 + 5) \bmod 13 = 8$

2. First substring hash (h_t) for "31415" = $(3 \cdot 10^4 + 1 \cdot 10^3 + 4 \cdot 10^2 + 1 \cdot 10^1 + 5) \bmod 13 = 10$

Slide the Pattern:

i. Compare h_p and h_t . If they match, check the characters.

ii. Compute the hash of the next substring using the rolling hash formula: $h_{t+1} = (d \cdot (h_t - T[i] \cdot d^{m-1}) + T[i+m]) \bmod q$

iii. For "14159": $h_t = (10 \cdot (10 - 3 \cdot 10^4) + 9) \bmod 13 = 8$.

3. Match Found:

i. When $h_p = h_t$, compare "26535" with the substring, confirming the match.

Time Complexity

Best Case: $O(n+m)$: If there are no hash collisions, the algorithm only computes hash values and performs $O(n)$ comparisons.

Worst Case: $O(n \cdot m)$: If all hash values collide, the algorithm falls back to brute force, comparing each substring character by character.

Average Case: $O(n+m)$: With a good hash function, collisions are rare, so the algorithm performs efficiently.

Q4: a) Explain the term Decision problem with the help of an example. Define the following problems and identify if they are decision problem or optimisation problem? Give reasons in support of your answer.

(i) Travelling Salesman Problem

(ii) Graph Colouring Problem

(iii) 0-1 Knapsack Problem

Ans. Decision Problem: A Decision Problem is a computational problem where the output is either "Yes" or "No" for a given input. These problems are concerned with determining whether a particular instance satisfies a certain property or condition.

Example:

Consider the problem of determining if a given number n is a prime number.

Input: An integer n .

Output: Yes (if n is a prime number) or No (if it is not).

Types of Problems

Decision Problem vs Optimization Problem

1. **Decision Problem:** Asks whether a solution exists that meets a specific criterion.
2. **Optimization Problem:** Asks for the "best" solution (e.g., minimum cost, maximum profit) among all feasible solutions.

Problem Analysis

(i) Travelling Salesman Problem (TSP)

Definition: Given a set of cities and distances between them, find the shortest possible route that visits each city exactly once and returns to the starting point.

Decision Problem:

Question: "Is there a route with a total cost (or distance) less than or equal to k ?"

Output: Yes or No.

Example: "Can we visit all cities with a total distance ≤ 100 ?"

Optimization Problem:

Goal: Minimize the total distance of the route.

Example: "What is the shortest route that visits all cities?"

Identification:

Decision Problem: If the problem only checks whether a route exists with cost $\leq k$, it is a decision problem.

Optimization Problem: Finding the shortest route makes it an optimization problem.

(ii) Graph Colouring Problem

Definition: Given a graph and k colors, assign colors to vertices so that no two adjacent vertices share the same color.

Decision Problem:

Question: "Can the graph be colored using k colors without two adjacent vertices having the same color?"

Output: Yes or No.

Example: "Can we color the graph with 3 colors?"

Optimization Problem:

Goal: Minimize the number of colors required to color the graph.

Example: "What is the minimum number of colors needed to color the graph?"

Identification:

Decision Problem: If the problem checks the feasibility of using k colors, it is a decision problem.

Optimization Problem: If the goal is to minimize the number of colors, it is an optimization problem.

(iii) 0-1 Knapsack Problem

Definition: Given n items with weights and values, and a knapsack of maximum capacity W , decide which items to include to maximize the total value without exceeding the capacity.

Decision Problem:

Question: "Is there a selection of items whose total weight is $\leq W$ and total value is $\geq V$?"

Output: Yes or No.

Example: "Can we achieve a value ≥ 100 with a capacity of 50?"

Optimization Problem:

Goal: Maximize the total value of items in the knapsack while staying within the weight capacity.

Example: "What is the maximum value we can achieve with a capacity of 50?"

Identification:

Decision Problem: If the problem asks whether a solution meeting certain constraints exists, it is a decision problem.

Optimization Problem: If the goal is to maximize the value, it is an optimization problem.

b) What are P and NP class of Problems? Explain each class with the help of at least two examples.

Ans. P and NP Classes of Problems: Computational problems are categorized into classes based on the resources (like time or space) required to solve them. Two important classes are P and NP.

1. Class P (Polynomial Time): Class P consists of problems that can be solved in polynomial time by a deterministic algorithm. This means that the time taken to solve the problem grows at most as a polynomial function of the input size n (e.g., n^2, n^3, n^k).

Key Characteristics:

- i. Problems in P are considered tractable or "easy to solve."
- ii. A solution can be found efficiently within polynomial time.

Examples:

1. **Sorting Problem (e.g., Merge Sort, Quick Sort):**
 - i. Input: A list of n elements.
 - ii. Task: Arrange the elements in ascending or descending order.
 - iii. Algorithm (e.g., Merge Sort) runs in $O(n \log n)$, which is polynomial time.
2. **Finding the Shortest Path (Dijkstra's Algorithm):**
 - i. Input: A weighted graph G with n vertices.
 - ii. Task: Find the shortest path from a source vertex to all other vertices.
 - iii. Dijkstra's algorithm runs in $O(n^2)$ for dense graphs, which is polynomial time.

2. Class NP (Nondeterministic Polynomial Time): Class NP consists of problems where a solution, if provided, can be verified in polynomial time by a deterministic algorithm. These problems may not be solvable in polynomial time, but verifying a solution is efficient.

Key Characteristics:

- i. Problems in NP are considered "easy to verify."
- ii. Many NP problems have no known polynomial-time algorithm for solving them.
- iii. All problems in P are also in NP because any problem that can be solved in polynomial time can also be verified in polynomial time.

Examples:

1. **Travelling Salesman Problem (TSP):**
 - i. Input: A set of n cities and distances between them.
 - ii. Task: Determine if there exists a route that visits all cities exactly once with a total distance $\leq k$.
 - iii. Verifying a given route and checking its total distance is polynomial, but finding the optimal route is computationally expensive.
2. **Subset Sum Problem:**
 - i. Input: A set of integers and a target sum S .
 - ii. Task: Determine if there exists a subset of integers that sums up to S .
 - iii. Verifying a subset (by adding up its elements) is polynomial, but finding the subset can take exponential time in the worst case.

c) Define the NP-Hard and NP-Complete problem. How are they different from each other. Explain the use of polynomial time reduction with the help of an example.

Ans. NP-Hard and NP-Complete Problems

1. NP-Hard Problems: A problem is NP-Hard if solving it in polynomial time would allow us to solve every problem in NP in polynomial time.

- i. It does not necessarily have to be a decision problem.
- ii. Verification in polynomial time is not required for NP-Hard problems.

Key Characteristics:

- i. NP-Hard problems are at least as hard as the hardest problems in NP.
- ii. They can be optimization or other non-decision problems.
- iii. Example: Travelling Salesman Problem (TSP) (find the shortest route that visits all cities).

2. NP-Complete Problems: A problem is NP-Complete if:

1. It belongs to the class NP (its solution can be verified in polynomial time).
2. It is NP-Hard (any NP problem can be reduced to it in polynomial time).

Key Characteristics:

- i. NP-Complete problems are both in NP and as hard as any problem in NP.
- ii. They must be decision problems.
- iii. Example: Boolean Satisfiability Problem (SAT) (determine if there exists an assignment of truth values to variables that satisfies a Boolean formula).

Difference Between NP-Hard and NP-Complete

Aspect	NP-Hard	NP-Complete
Class Membership	Not necessarily in NP.	Must belong to NP.
Verification in Polynomial Time	Not required.	Required.
Problem Type	Can be decision, optimization, or other problems.	Must be a decision problem.

Reduction	Any NP problem can be reduced to it in polynomial time.	Any NP problem can be reduced to it in polynomial time.
Example	Travelling Salesman Problem (optimization form).	Boolean Satisfiability Problem (SAT).

Polynomial-Time Reduction

Definition: Polynomial-time reduction is a method used to transform one problem into another in polynomial time. If problem A can be reduced to problem BBB in polynomial time and BBB is known to be solvable in polynomial time, then A is also solvable in polynomial time.

Purpose: Polynomial-time reduction is used to show that a problem is at least as hard as another problem. It is also used to classify problems as NP-Complete or NP-Hard.

Example of Polynomial-Time Reduction

Reduction of 3-SAT to Clique Problem:

1. 3-SAT Problem:

- Input: A Boolean formula in Conjunctive Normal Form (CNF) with clauses of exactly three literals.
- Output: Determine if there exists an assignment of truth values to variables that satisfies the formula.
- Known to be NP-Complete.

2. Clique Problem:

- Input: A graph $G=(V,E)$ and an integer k .
- Output: Determine if there exists a clique (a subset of k vertices where every pair of vertices is connected by an edge).
- Known to be NP-Complete.

Reduction Steps:

- For each clause in the 3-SAT formula, create a set of vertices representing its literals.
- Add edges between all pairs of vertices from different clauses if their literals do not contradict each other.
- Solve the Clique problem for k , where k is the number of clauses in the formula.
- If the Clique exists, it corresponds to a satisfying assignment for the 3-SAT formula.

d) Define the following Problems:

(i) SAT Problem

(ii) Clique problem

(iii) Hamiltonian Cycle Problem

(iv) Subset Sum Problem

Ans.

(i) SAT Problem

The SAT problem is the problem of determining whether there exists an assignment of truth values (true/false) to the variables of a given Boolean formula such that the entire formula evaluates to true.

Input: A Boolean formula in Conjunctive Normal Form (CNF), i.e., a conjunction (AND) of disjunctions (OR) of literals (variables or their negations).

Example: $(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2)$.

Output: True if a satisfying assignment exists, False otherwise.

Complexity: The SAT problem was the first problem proven to be NP-Complete (Cook's Theorem).

Example:

For the formula $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$:

A satisfying assignment is $x_1 = \text{True}, x_2 = \text{False}, x_3 = \text{True}$.

(ii) Clique Problem: The Clique problem is the problem of finding a subset of vertices in a graph that forms a clique (i.e., a complete subgraph where every pair of vertices is connected).

Input: A graph $G=(V,E)$ and an integer k .

Output: True if there exists a subset of k vertices in G such that every two vertices in the subset are connected, False otherwise.

Complexity: The decision version of the Clique problem is NP-Complete.

Example:

For a graph with vertices $\{A,B,C,D\}$ and edges $\{(A,B),(B,C),(C,D),(A,C)\}$:

A clique of size 3 is $\{A,B,C\}$.

(iii) Hamiltonian Cycle Problem: The Hamiltonian Cycle problem is the problem of determining whether there exists a cycle in a graph that visits each vertex exactly once and returns to the starting vertex.

Input:

A graph $G=(V,E)$.

Output: True if there exists a Hamiltonian cycle in G , False otherwise.

Complexity: The Hamiltonian Cycle problem is NP-Complete.

Example:

For a graph with vertices $\{A,B,C,D\}$ and edges $\{(A,B),(B,C),(C,D),(D,A),(A,C)\}$:

A Hamiltonian cycle is $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$.

(iv) Subset Sum Problem: The Subset Sum problem is the problem of determining whether there exists a subset of a given set of integers whose sum is equal to a specified target value.

Input: A set of integers $S=\{a_1, a_2, \dots, a_n\}$ and a target integer T .

Output: True if there exists a subset $S' \subseteq S$ such that $\sum_{x \in S'} x = T$, False otherwise.

Complexity: The decision version of the Subset Sum problem is NP-Complete.

Example:

For $S=\{3,4,5,12\}$ and $T=9$:

A valid subset is $\{4,5\}$ because $4+5=9$.